נספח ט': מדריך מימוש GateKeeper

ט.1 מבוא

נספח זה מספק הנחיות מפורטות למימוש תבנית GateKeeper בסוכני שחקן ושופט. התבנית מגנה על הסוכן מפני חריגה ממגבלות Gmail API ומפני לולאות אינסופיות.

ט.1.1 למה הסוכן צריך GateKeeper?

- **מגבלות Gmail API** — 500 הודעות ביום (מומלץ: 400)
- **מגבלת קצב** — 500 קריאות API בדקה
- **מניעת לולאות** — באג בקוד עלול לשלוח מאות הודעות בדקות
- **הגנה עצמית** — הסוכן עלול "לתקוף" את עצמו בטעות

חובת מימוש

מימוש GateKeeper הוא **חובה** בפרויקט. סוכן שחורג ממגבלות עלול להיחסם על ידי Google ולפגוע בציון.

ט.2 רכיבי ה-GateKeeper

תבנית GateKeeper מורכבת משלושה רכיבים עיקריים:

טבלה 208: רכיבי GateKeeper

רכיב	תפקיד
QuotaManager	ניהול מכסה יומית (400 הודעות)
RateLimiter	הגבלת קצב שליחה (Token Bucket)
DOSDetector	זיהוי לולאות אינסופיות

ט.3 מימוש QuotaManager

מנהל המכסות עוקב אחר מספר ההודעות שנשלחו ביום ומתריע כאשר מתקרבים למגבלה.

QuotaManager מחלקת

```
from datetime import datetime, date
from dataclasses import dataclass
import logging

@dataclass
class QuotaState:
    daily_count: int = 0
    last_reset_date: date = None

class QuotaManager:
    """Manages daily email quota to prevent Gmail API limit violations."
   ""

    def __init__(self, daily_limit: int = 400, warning_threshold: float
   = 0.8):
        self.daily_limit = daily_limit
        self.warning_threshold = warning_threshold
        self.state = QuotaState()
        self.logger = logging.getLogger(__name__)

    def can_send(self) -> bool:
        """Check if we can send another email."""
        self._check_daily_reset()
        return self.state.daily_count < self.daily_limit

    def record_send(self) -> None:
        """Record that an email was sent."""
        self._check_daily_reset()
        self.state.daily_count += 1

        # Warn if approaching limit
        if self.state.daily_count >= self.daily_limit * self.
   warning_threshold:
            remaining = self.daily_limit - self.state.daily_count
            self.logger.warning(f"Quota warning: {remaining} emails
   remaining today")

    def _check_daily_reset(self) -> None:
        """Reset counter at midnight."""
        today = date.today()
        if self.state.last_reset_date != today:
            self.state.daily_count = 0
            self.state.last_reset_date = today
            self.logger.info("Daily quota reset")
```

```
    def get_remaining(self) -> int:
        """Get number of emails remaining today."""
        self._check_daily_reset()
        return self.daily_limit - self.state.daily_count
```

ט.4 מימוש RateLimiter (Token Bucket)

אלגוריתם Token Bucket מאפשר שליחה בפרצים תוך שמירה על קצב ממוצע.

ט.4.1 עקרון הפעולה

- דלי מכיל tokens (קיבולת מקסימלית)
- כל שליחת הודעה צורכת token אחד
- tokens מתמלאים בקצב קבוע
- אם אין tokens, השליחה נחסמת עד שיהיו

מחלקת TokenBucket

```
import time
from threading import Lock

class TokenBucket:
    """Token bucket rate limiter for Gmail API calls."""

    def __init__(self, rate: float = 1.0, capacity: int = 10):
        """
        Args:
            rate: Tokens added per second (e.g., 1.0 = 1 token/sec)
            capacity: Maximum tokens in bucket (burst capacity)
        """
        self.rate = rate
        self.capacity = capacity
        self.tokens = capacity
        self.last_refill = time.monotonic()
        self.lock = Lock()

    def acquire(self, tokens: int = 1, blocking: bool = True) -> bool:
        """Acquire tokens, optionally blocking until available."""
        with self.lock:
            self._refill()

            if self.tokens >= tokens:
```


© Dr. Segal Yoram - כל הזכויות שמורות

```
                self.tokens -= tokens
                return True

            if not blocking:
                return False

            # Calculate wait time
            needed = tokens - self.tokens
            wait_time = needed / self.rate

        # Wait outside lock
        time.sleep(wait_time)

        # Retry after waiting
        with self.lock:
            self._refill()
            if self.tokens >= tokens:
                self.tokens -= tokens
                return True
        return False

    def _refill(self) -> None:
        """Refill tokens based on elapsed time."""
        now = time.monotonic()
        elapsed = now - self.last_refill
        new_tokens = elapsed * self.rate
        self.tokens = min(self.capacity, self.tokens + new_tokens)
        self.last_refill = now
```

טבלה 209: פרמטרים מומלצים ל-Token Bucket

תרחיש	קצב (rate)	קיבולת (capacity)
פעילות רגילה	1.0	10
משחק פעיל	2.0	15
שידורים	0.5	5

5.ט מימוש DOSDetector

מזהה דפוסי שליחה חריגים שמעידים על באג או לולאה אינסופית.

מחלקת DOSDetector

```
from collections import deque
from datetime import datetime, timedelta
import logging

class DOSDetector:
    """Detects abnormal sending patterns that indicate bugs."""

    def __init__(self, max_per_minute: int = 5, window_seconds: int =
    60):
        self.max_per_minute = max_per_minute
        self.window = timedelta(seconds=window_seconds)
        self.send_times: deque[datetime] = deque()
        self.logger = logging.getLogger(__name__)
        self.circuit_open = False

    def record_send(self) -> None:
        """Record a send event and check for DOS pattern."""
        now = datetime.now()
        self.send_times.append(now)
        self._cleanup_old_entries(now)

        if len(self.send_times) > self.max_per_minute:
            self.circuit_open = True
            self.logger.critical(
                f"DOS detected! {len(self.send_times)} sends in {self.
    window.seconds}s. "
                f"Circuit breaker OPEN."
            )

    def is_blocked(self) -> bool:
        """Check if sending is blocked due to DOS detection."""
        return self.circuit_open

    def reset(self) -> None:
        """Manual reset after fixing the issue."""
        self.circuit_open = False
        self.send_times.clear()
        self.logger.info("DOS detector reset")

    def _cleanup_old_entries(self, now: datetime) -> None:
        """Remove entries outside the time window."""
        cutoff = now - self.window
        while self.send_times and self.send_times[0] < cutoff:
            self.send_times.popleft()
```


© Dr. Segal Yoram - כל הזכויות שמורות

Circuit Breaker

כאשר DOSDetector מזהה דפוס חריג, הוא פותח את ה-Circuit Breaker וחוסם את כל השליחות. נדרשת התערבות ידנית (או אוטומטית לאחר זמן קירור) לאיפוס.

ט.6 מימוש GateKeeper המשולב

כעת נשלב את שלושת הרכיבים למחלקה אחת:

מחלקת GateKeeper המשולבת

```
class GateKeeper:
    """Combined gate keeper for email sending protection."""

    def __init__(self):
        self.quota = QuotaManager(daily_limit=400)
        self.rate_limiter = TokenBucket(rate=1.0, capacity=10)
        self.dos_detector = DOSDetector(max_per_minute=5)

    def can_send(self) -> tuple[bool, str]:
        """Check all gates before sending."""
        # Check DOS first (highest priority)
        if self.dos_detector.is_blocked():
            return False, "DOS_BLOCKED"

        # Check daily quota
        if not self.quota.can_send():
            return False, "QUOTA_EXCEEDED"

        # Check rate limit (may block)
        if not self.rate_limiter.acquire(blocking=False):
            return False, "RATE_LIMITED"

        return True, "OK"

    def record_send(self) -> None:
        """Record successful send across all components."""
        self.quota.record_send()
        self.dos_detector.record_send()

    def send_email(self, email_func, *args, **kwargs):
        """Wrapper that applies all gates before sending."""
        can_send, reason = self.can_send()
        if not can_send:
            raise GateKeeperBlockedError(reason)
```

```
        # Wait for rate limit if needed
        self.rate_limiter.acquire(blocking=True)

        # Send the email
        result = email_func(*args, **kwargs)

        # Record successful send
        self.record_send()
        return result

class GateKeeperBlockedError(Exception):
    """Raised when GateKeeper blocks a send operation."""
    pass
```

7.ט שילוב עם סוכן השחקן

שילוב GateKeeper בסוכן

```
class PlayerAgent:
    def __init__(self):
        self.gmail_client = GmailClient()
        self.gatekeeper = GateKeeper()

    async def send_message(self, msg: dict) -> bool:
        """Send message through gatekeeper."""
        try:
            return self.gatekeeper.send_email(
                self.gmail_client.send,
                msg
            )
        except GateKeeperBlockedError as e:
            self.logger.error(f"Send blocked: {e}")
            if str(e) == "QUOTA_EXCEEDED":
                # Wait until tomorrow
                await self._wait_for_quota_reset()
            elif str(e) == "DOS_BLOCKED":
                # Alert and halt
                await self._alert_admin("DOS detected!")
                raise
            return False
```


© Dr. Segal Yoram - כל הזכויות שמורות

8.ט רשימת בדיקה למימוש

- ☐ QuotaManager מותקן עם מגבלה של 400 הודעות
- ☐ TokenBucket מותקן עם פרמטרים מתאימים
- ☐ DOSDetector מותקן עם סף של 5 הודעות לדקה
- ☐ כל שליחת הודעה עוברת דרך GateKeeper
- ☐ לוגים מתועדים כאשר מגיעים ל-80% מהמכסה
- ☐ Circuit Breaker פועל בזיהוי DOS
- ☐ קיים מנגנון התאוששות לאחר חסימה